

ECE1778: Creative Applications for Mobile Devices

Chen Ying

Assistant Professor, Teaching Stream

Department of Electrical and Computer Engineering

University of Toronto

Last Week's Lecture

Redux Toolkit: Scalable global state for larger apps

- **Slices:** Organize state + reducers + actions
- **Provider + hooks:** Any component can read or update state using `useSelector`, `useDispatch`
- **Immer:** Enables “mutating” syntax for immutable updates

Last Week's Lecture

Data Persistence

- **React Native Async Storage:** Asynchronous, persistent key-value storage for React Native
- **API:** `setItem`, `getItem`, `removeItem`, ...
- **Error handling:** `try/catch`
- **Integration with Redux:** Load on app start (`app/_layout.tsx`); save after updates (reducers)
- **Expo SecureStore:** Encrypted storage for iOS and Android

Example: Async Storage with Redux

| async-storage-demo/feature/theme/themeSlice.ts

```
1 import { createSlice, type PayloadAction } from "@reduxjs/toolkit";
2 import { saveTheme } from "../../storage/themeStorage";
3
4 type ThemeState = { theme: "light" | "dark" };
5
6 const initialState: ThemeState = { theme: "light" };
7
8 const themeSlice = createSlice({
9   name: "theme",
10  initialState,
11  reducers: {
12    setTheme: (state, action: PayloadAction<"light" | "dark">) => {
13      state.theme = action.payload;
14      saveTheme(action.payload);
15    },
16    toggleTheme: (state) => {
17      const newTheme = state.theme === "light" ? "dark" : "light";
18      state.theme = newTheme;
19      saveTheme(newTheme);
```

No `await` in Reducers

| async-storage-demo/feature/theme/themeSlice.ts

```
const themeSlice = createSlice({
  reducers: {
    setTheme: (state, action: PayloadAction<"light" | "dark">) => {
      state.theme = action.payload;
      saveTheme(action.payload);
    },
    toggleTheme: (state) => {
      const newTheme = state.theme === "light" ? "dark" : "light";
      state.theme = newTheme;
      saveTheme(newTheme);
    },
  },
});
```

saveTheme is async

| async-storage-demo/storage/themeStorage.ts

```
export async function saveTheme(theme: "light" | "dark"): Promise<void> {}
```

No `await` in Reducers

Reducers in **Redux** (and Redux Toolkit) must be **synchronous**

- Reducers should only update state
- Take current state + action → Return new state
- Redux relies on deterministic behavior
- Same input → Same output
- Async operations break this guarantee
- Reducers should **not** contain side effects
- No `await`, no I/O, no API calls...

Why `setTheme` Works

```
setTheme: (state, action) => {  
  state.theme = action.payload;  
  saveTheme(action.payload);  
}
```

- `saveTheme()` is simply called and runs in the background
 - Redux state updates immediately
 - UI stays responsive — persistence happens later
- This pattern is fine for **simple side effects**, like storing UI preferences
- App doesn't depend on async result
 - Redux state remains correct even if storage fails

Strict Redux Rule

Ideally, side effects should live outside reducers

- Use **thunks** for async logic
- **Thunk**: “a piece of code that does some delayed work”
- Use **middleware** for automatic side effects
- **Middleware**: Code that observes actions as they are dispatched, allowing side effects before they reach the reducers

Use Thunks

In programming, a **thunk** is a function that delays or wraps a computation inside another function

In **Redux**, thunks are used to handle **asynchronous logic** (like API calls or Async Storage) **outside reducers**

- A **thunk function**: An **action creator** that returns another function that receives `dispatch` (optionally `getState`)

Redux Thunk Function

An **action creator** that returns another function that receives `dispatch` (optionally `getState`)

Example with raw thunk pattern

```
export const setThemeAsync = (theme) => {  
  return async (dispatch) => {  
    await saveTheme(theme);           // async work  
    dispatch(setTheme(theme));        // update Redux state  
  };  
};
```

A thunk function allows you to

- Perform asynchronous work
- Dispatch actions before or after the async work

Create Thunks

Redux Toolkit provides `createAsyncThunk()` to simplify writing thunk functions

- No need to call `dispatch` manually
- Automatically generates action types for
 - `pending`: Before async work starts
 - `fulfilled`: After success
 - `rejected`: After failure

Create Thunk Function

A thunk function created by `createAsyncThunk()`

- Runs async code inside (e.g., `await saveTheme(theme)`)
- Dispatches actions for each async lifecycle stage

```
export const setThemeAsync = createAsyncThunk(
  "theme/saveTheme",
  async (theme: "light" | "dark") => {
    await saveTheme(theme);
    return theme; // returned value updates Redux state in 'fulfilled' case
  }
);
```

Official Doc: [createAsyncThunk](#)

Use Thunks

```
features/theme/  
├─ themeSlice.ts      // slice + reducers  
└─ themeThunks.ts     // async thunks like setThemeAsync, loadThemeAsync
```

Create Thunks

| async-storage-demo/feature/theme/themeThunks.ts

```
1 import { createAsyncThunk } from "@reduxjs/toolkit";
2 import { loadTheme, saveTheme } from "../../storage/themeStorage";
3 import { RootState } from "../../store/store";
4
5 export const loadThemeAsync = createAsyncThunk("theme/loadTheme", async (
6   const theme = await loadTheme();
7   return theme;
8 });
9
10 export const setThemeAsync = createAsyncThunk(
11   "theme/setThemeAsync",
12   async (theme: "light" | "dark") => {
13     await saveTheme(theme);
14     return theme;
15   }
16 );
17
18 export const toggleThemeAsync = createAsyncThunk(
19   "theme/toggleTheme",
```

- Access latest state via `getState`

Handle in Slice

| async-storage-demo/feature/theme/themeSlice.ts

```
1 import { createSlice, type PayloadAction } from "@reduxjs/toolkit";
2 import { loadThemeAsync, setThemeAsync, toggleThemeAsync } from "../themeT
3
4 type ThemeState = { theme: "light" | "dark" };
5
6 const initialState: ThemeState = { theme: "light" };
7
8 const themeSlice = createSlice({
9   name: "theme",
10  initialState,
11  reducers: {
12    setTheme: (state, action: PayloadAction<"light" | "dark">) => {
13      state.theme = action.payload;
14    },
15    toggleTheme: (state) => {
16      const newTheme = state.theme === "light" ? "dark" : "light";
17      state.theme = newTheme;
18    },
19  },
```

extraReducers handle actions generated by thunks

- Internal to the slice — no need to export

Handle in Slice

| async-storage-demo/feature/theme/themeSlice.ts

```
1 import { createSlice, type PayloadAction } from "@reduxjs/toolkit";
2 import { loadThemeAsync, setThemeAsync, toggleThemeAsync } from "../themeT
3
4 type ThemeState = { theme: "light" | "dark" };
5
6 const initialState: ThemeState = { theme: "light" };
7
8 const themeSlice = createSlice({
9   name: "theme",
10  initialState,
11  reducers: {
12    setTheme: (state, action: PayloadAction<"light" | "dark">) => {
13      state.theme = action.payload;
14    },
15    toggleTheme: (state) => {
16      const newTheme = state.theme === "light" ? "dark" : "light";
17      state.theme = newTheme;
18    },
19  },
```

Keep both sync and async actions for flexibility

- `setTheme` / `toggleTheme` for immediate UI updates
- `setThemeAsync` / `toggleThemeAsync` for update + persist

Handle in Slice

async-storage-demo/feature/theme/themeSlice.ts

```
1 import { createSlice, type PayloadAction } from "@reduxjs/toolkit";
2 import { loadThemeAsync, setThemeAsync, toggleThemeAsync } from "../themeT
3
4 type ThemeState = { theme: "light" | "dark" };
5
6 const initialState: ThemeState = { theme: "light" };
7
8 const themeSlice = createSlice({
9   name: "theme",
10  initialState,
11  reducers: {
12    setTheme: (state, action: PayloadAction<"light" | "dark">) => {
13      state.theme = action.payload;
14    },
15    toggleTheme: (state) => {
16      const newTheme = state.theme === "light" ? "dark" : "light";
17      state.theme = newTheme;
18    },
19  },
```

Reducers remain pure and synchronous

All async side effects live in `themeThunks.ts`

Update app/_layout.tsx

async-storage-demo/app/_layout.tsx

```
1 import { useEffect } from "react";
2 import { Stack } from "expo-router";
3 import { Provider } from "react-redux";
4 import { store } from "../store/store";
5 import { loadThemeAsync } from "../features/theme/themeThunks";
6
7 export default function RootLayout() {
8   useEffect(() => {
9     store.dispatch(loadThemeAsync());
10  }, []);
11
12  return (
13    <Provider store={store}>
14      <Stack />
15    </Provider>
16  );
17 }
```

Update `app/index.tsx`

`async-storage-demo/app/index.tsx`

```
1 import { View, Text, Button, StyleSheet } from "react-native";
2 import { useSelector, useDispatch } from "react-redux";
3 import { RootState, AppDispatch } from "../store/store";
4 import { toggleThemeAsync } from "../features/theme/themeThunks";
5
6 export default function HomeScreen() {
7   const theme = useSelector((state: RootState) => state.theme.theme);
8   const dispatch: AppDispatch = useDispatch();
9
10  return (
11    <View
12      style={[styles.container, theme === "light" ? styles.light : styles
13    >
14      <Text style={styles.text}>Current theme: {theme}</Text>
15      <Button
16        title="Toggle Theme"
17        onPress={() => dispatch(toggleThemeAsync())}
18      />
19    </View>
```

Use Middleware

Middleware can observe actions automatically and run side effects

Listener Middleware: Modern Redux Toolkit approach

- Watches actions as they are dispatched
- Runs async logic without blocking reducers
- Keeps reducers pure and simple

“Automatic side-effect handlers”

Create Middleware

Redux Toolkit provides `createListenerMiddleware()`

middleware-demo/store/store.ts

```
1 import { createListenerMiddleware } from "@reduxjs/toolkit";
2 import { setTheme } from "../features/theme/themeSlice";
3 import { saveTheme } from "../storage/themeStorage";
4
5 export const listenerMiddleware = createListenerMiddleware();
6
7 listenerMiddleware.startListening({
8   actionCreator: setTheme,
9   effect: async (action) => {
10     await saveTheme(action.payload);
11   },
12 });
```

Official Doc: [createListenerMiddleware](#)

Add to Store

Add listener middleware to store

middleware-demo/store/store.ts

```
export const store = configureStore({
  reducer: {
    theme: themeReducer,
  },
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware().prepend(listenerMiddleware.middleware),
});
```

- Reducers stay pure
- Persistence happens automatically whenever `setTheme` is dispatched

Reducers Stay Pure

middleware-demo/features/theme/themeSlice.ts

```
1 import { createSlice, type PayloadAction } from "@reduxjs/toolkit";
2 type ThemeState = { theme: "light" | "dark" };
3
4 const initialState: ThemeState = { theme: "light" };
5
6 const themeSlice = createSlice({
7   name: "theme",
8   initialState,
9   reducers: {
10     setTheme: (state, action: PayloadAction<"light" | "dark">) => {
11       state.theme = action.payload;
12     },
13     toggleTheme: (state) => {
14       const newTheme = state.theme === "light" ? "dark" : "light";
15       state.theme = newTheme;
16     },
17   },
18 });
19
```

Update `app/_layout.tsx`

middleware-demo/app/_layout.tsx

```
1 import { useEffect } from "react";
2 import { Stack } from "expo-router";
3 import { Provider } from "react-redux";
4 import { store } from "../store/store";
5 import { setTheme } from "../features/theme/themeSlice";
6 import { loadTheme } from "../storage/themeStorage";
7
8 export default function RootLayout() {
9   useEffect(() => {
10     loadTheme().then((theme) => {
11       if (theme) {
12         store.dispatch(setTheme(theme)); // triggers middleware automatic
13       }
14     });
15   }, []);
16
17   return (
18     <Provider store={store}>
19     <Stack />
```

Middleware listens for `setTheme` and automatically saves it to Async Storage

- No need to call `saveTheme()` manually

Update `app/index.tsx`

middleware-demo/app/index.tsx

```
1 import { View, Text, Button, StyleSheet } from "react-native";
2 import { useSelector, useDispatch } from "react-redux";
3 import { RootState, AppDispatch } from "../store/store";
4 import { toggleTheme } from "../features/theme/themeSlice";
5
6 export default function HomeScreen() {
7   const theme = useSelector((state: RootState) => state.theme.theme);
8   const dispatch: AppDispatch = useDispatch();
9
10  return (
11    <View
12      style={[styles.container, theme === "light" ? styles.light : styles
13    >
14      <Text style={styles.text}>Current theme: {theme}</Text>
15      <Button title="Toggle Theme" onPress={() => dispatch(toggleTheme())
16    </View>
17  );
18 }
19
```

`toggleTheme` also persists to Async Storage automatically

- Add another listener in `store/store.ts`

Add Another Listener

middleware-demo/store/store.ts

```
1 import { createListenerMiddleware, configureStore } from "@reduxjs/toolkit";
2 import themeReducer, {
3   setTheme,
4   toggleTheme,
5 } from "../features/theme/themeSlice";
6 import { saveTheme } from "../storage/themeStorage";
7
8 export const listenerMiddleware = createListenerMiddleware();
9
10 // Save theme when setTheme is dispatched
11 listenerMiddleware.startListening({
12   actionCreator: setTheme,
13   effect: async (action) => {
14     await saveTheme(action.payload);
15   },
16 });
17
18 // Save theme when toggleTheme is dispatched
19 listenerMiddleware.startListening({
```

`listenerApi.getState()` returns the latest Redux state after the reducer has run

- Use it when the action does not carry the value you need

Recap: Handle Async Side Effects in Redux

No `await` in reducers

- Reducers must be synchronous and pure
- Async work (I/O, API calls, etc.) breaks determinism
- Reducers only update states
- Side effects go elsewhere
- **Thunks**
- **Middleware**

Recap: Thunks

Thunk: A function that delays logic for later execution

Redux Thunk: An action creator that returns a function receiving `dispatch` (and optionally `getState`)

- Performs async work
- Dispatches actions before or after async work

Redux Toolkit simplifies this with `createAsyncThunk()`

- Automatically generates `pending`, `fulfilled`, `rejected` actions
- Async logic lives outside reducers

Recap: Middleware

Code that observes dispatched actions and can run side effects automatically

Listener Middleware (Redux Toolkit)

- Watches for specific actions (`setTheme`, `toggleTheme`)
- Runs async logic after reducers have updated state
- Example: `listenerApi.getState()` gives the latest state after the action

Thunks & Middleware

Both handle async work while keeping reducers pure and synchronous

- **Thunks:** Action creators that return a function
 - Receive `dispatch` (and optionally `getState`)
 - Perform async work and dispatch actions before/after
- **Middleware:** Automatic side-effect handlers
 - Observes actions automatically as they are dispatched
 - Can run async logic without blocking reducers

Redux Toolkit provides built-in utilities for both patterns

Thunks vs. Middleware

Choose the right tool based on **who triggers the async work** and **how automatic it should be**

- **Thunks:** Triggered explicitly by calling a thunk action
 - Great for async logic tied to a specific user action
 - Can access latest state via `getState`
- **Middleware:** Triggered automatically whenever a specific action is dispatched
 - Ideal for automatic side effects
 - Access updated state via `listenerApi.getState()`

Today's Lecture

Handling Async Side Effects in Redux
Notifications

Notifications

Notifications inform users even when the app is inactive

Two Types

- **Local Notifications:** Scheduled and triggered directly by the device
- **Push Notifications:** Delivered from a remote server via the internet

Local vs. Push Notifications

Type	Triggered By	Internet Required	Typical Use
Local	App (device-side)	No	Reminders, timers, offline alerts
Push	Server or backend	Yes	Chat messages, live updates

Expo Notifications

Expo provides a cross-platform API for both local and push notifications

- Official Doc: **Expo Notifications**

Key Features

- Zero native setup for local notifications
- Instant testing in Expo Go
- Requires minimal permissions handling

Set Up

Install package

```
npx expo install expo-notifications
```

Set Notification Handler

| local-notification-demo/App.tsx

```
1 import * as Notifications from "expo-notifications";
2
3 Notifications.setNotificationHandler({
4   handleNotification: async () => ({
5     shouldShowBanner: true, // Show notification as a banner/alert
6     shouldShowList: true, // Add notification to system list/center for l
7     shouldPlaySound: true, // Play sound on arrival
8     shouldSetBadge: false, // Do not update app icon badge count
9   }),
10 });
```

`handleNotification` is called when a notification is received

Official Doc: [setNotificationHandler](#)

Schedule Notification

Use `scheduleNotificationAsync()` to schedule a notification to be triggered in the future

| local-notification-demo/App.tsx

```
Notifications.scheduleNotificationAsync({  
  content: { title: "Daily Reminder", body: "Time to check your app!" },  
  trigger: {  
    type: Notifications.SchedulableTriggerInputTypes.TIME_INTERVAL,  
    seconds: 5,  
  },  
});
```

Official Doc: [scheduleNotificationAsync](#)

Live Demo

| local-notification-demo/App.tsx

```
1 import * as Notifications from "expo-notifications";
2 import { View, StyleSheet } from "react-native";
3
4 Notifications.setNotificationHandler({
5   handleNotification: async () => ({
6     shouldPlaySound: false,
7     shouldSetBadge: false,
8     shouldShowBanner: true,
9     shouldShowList: true,
10   }),
11 });
12
13 Notifications.scheduleNotificationAsync({
14   content: { title: "Daily Reminder", body: "Time to check your app!" },
15   trigger: {
16     type: Notifications.SchedulableTriggerInputTypes.TIME_INTERVAL,
17     seconds: 5,
18   },
19 });
```

Scheduling fails silently without user permission

Always request user permission first

Request Permission

Prompt for permissions early (e.g., on app load) using `requestPermissionsAsync()`

| local-notification-demo/App.tsx

```
1 export default function App() {
2   useEffect(() => {
3     registerForNotifications();
4   }, []);
5
6   async function registerForNotifications() {
7     const { status } = await Notifications.requestPermissionsAsync();
8     if (status !== "granted") {
9       Alert.alert("Permission not granted!");
10    }
11    return;
12  }
13 }
```

Live Demo

| local-notification-demo/App.tsx

```
1 import * as Notifications from "expo-notifications";
2 import { useEffect } from "react";
3 import { Alert, View, StyleSheet } from "react-native";
4
5 Notifications.setNotificationHandler({
6   handleNotification: async () => ({
7     shouldPlaySound: false,
8     shouldSetBadge: false,
9     shouldShowBanner: true,
10    shouldShowList: true,
11  }),
12 });
13
14 Notifications.scheduleNotificationAsync({
15   content: { title: "Daily Reminder", body: "Time to check your app!" },
16   trigger: {
17     type: Notifications.SchedulableTriggerInputTypes.TIME_INTERVAL,
18     seconds: 5,
19   },
```

Best Practice: Request permissions only once, on first app launch

Add A Schedule Button

Add a “Schedule Notification” button to trigger notifications

- User taps → Notification schedules → Display after 5 seconds

Add A Schedule Button

| local-notification-demo/App.tsx

```
1 import * as Notifications from "expo-notifications";
2 import { useEffect } from "react";
3 import { Alert, Button, View, StyleSheet } from "react-native";
4
5 Notifications.setNotificationHandler({
6   handleNotification: async () => ({
7     shouldShowBanner: true,
8     shouldShowList: true,
9     shouldPlaySound: true,
10    shouldSetBadge: false,
11  }),
12 });
13
14 export default function App() {
15   useEffect(() => {
16     registerForNotifications();
17   }, []);
18
19   async function registerForNotifications() {
```

Handle Notification Events

Listen for user interaction with

`addNotificationResponseReceivedListener`

- Handle user taps

```
Notifications.addNotificationResponseReceivedListener((response) => {  
    console.log("User tapped notification:", response.notification.request.cont  
    // Navigate, update state, etc.  
});
```

Official Doc: [Listen to Notification Events](#)

Add Event Listener

| local-notification-demo/App.tsx

```
1 export default function App() {
2   useEffect(() => {
3     registerForNotifications();
4   }, []);
5
6   useEffect(() => {
7     const subscription = Notifications.addNotificationResponseReceivedLis
8       (response) => {
9         console.log(
10           "User tapped notification:",
11           response.notification.request.content
12         );
13       }
14     );
15
16     return () => subscription.remove();
17   }, []);
18 }
```

subscription: An event listener that listens for any user tap on any incoming notification while your app is running (foreground or background)

Add Event Listener

| local-notification-demo/App.tsx

```
1 export default function App() {
2   useEffect(() => {
3     registerForNotifications();
4   }, []);
5
6   useEffect(() => {
7     const subscription = Notifications.addNotificationResponseReceivedLis
8       (response) => {
9         console.log(
10           "User tapped notification:",
11           response.notification.request.content
12         );
13       }
14     );
15
16     return () => subscription.remove();
17   }, []);
18 }
```

`subscription.remove()`: Clean up on unmount to avoid duplicates/memory leaks

Full Implementation

| local-notification-demo/App.tsx

```
1 import * as Notifications from "expo-notifications";
2 import { useEffect } from "react";
3 import { Alert, Button, View, StyleSheet } from "react-native";
4
5 Notifications.setNotificationHandler({
6   handleNotification: async () => ({
7     shouldShowBanner: true,
8     shouldShowList: true,
9     shouldPlaySound: true,
10    shouldSetBadge: false,
11  }),
12 });
13
14 export default function App() {
15   useEffect(() => {
16     registerForNotifications();
17   }, []);
18
19   useEffect(() => {
```

Local Notifications

Ideal for offline, device-driven scenarios

Common Use Cases

- **Daily Reminders:** Habit trackers, medication alerts
- **Timers & Schedules:** Workout sessions, study breaks
- **Event Alerts:** Local countdowns, offline to-dos

All powered by the device — no backend or internet required

Recap: Local Notification

Device-scheduled alerts (vs. server-pushed)

Implement with Expo Notifications

- Core Steps
 - Request permissions: `requestPermissionsAsync`
 - Schedule: `scheduleNotificationAsync`
 - Handle user interactions:
`addNotificationResponseReceivedListener`
- Best Practices: Request permissions once; clean up listeners

Push Notifications

Push notifications are **sent from a backend service** to user devices — even when the app is closed

- Require device registration & push tokens for targeting
- Delivered via a push provider like **FCM (Firebase Cloud Messaging)** or **APNs (Apple Push Notification Service)**.
- Useful for remote updates, real-time messages, timely alerts

Push Providers

- **Expo Push Notifications**: Simplified API for sending push notifications
 - Uses FCM (Android) and APNs (iOS) under the hood
 - Best for Expo projects
- **Firestore Cloud Messaging (FCM)**: Feature-rich push delivery platform by Google
 - Directly supports Android, iOS, and web
 - Works with both native and Expo apps
- **Apple Push Notification Service (APNs)**

Expo Push Notifications

Expo streamlines push notifications across platforms

- Manages integration with Firebase Cloud Messaging (FCM) for Android or Apple Push Notification service (APNs) for iOS
- Uses Expo's cloud service (Expo Push Service) for reliable delivery — even when the app is closed

Official Doc: **Expo Push Notifications Setup**

Expo Push Notifications

Key Requirement: Push token ([ExpoPushToken](#))

- A unique ID tying your device to the Expo project
- Acts as an “address” for your server to target and send notifications via Expo

Your Server

The **server** is your app's remote backend (e.g., Node.js app on a cloud host)

- Bridges the app and devices: Securely routes pushes without direct device-to-device contact
- **Role**
 - Stores user data (e.g., push tokens)
 - Detects events (e.g., new message)
 - Sends notifications via Expo/FCM/APNs

Example: Chat App

1. User A sends a message → Your server receives it
2. Server looks up User B's push token (stored in database)
3. Server tells Expo: "Send alert to token XYZ: 'New message from A'"
4. Expo delivers to User B's device → Notification appears

Expo Push Notifications

Key Requirement: Push token ([ExpoPushToken](#))

- A unique ID tying your device to the Expo project
- Acts as an “address” for your server to target and send notifications via Expo

Testing Note: Requires a physical device

- Simulators/emulators unsupported

Setup: Install Libraries

Add core packages for tokens, permissions, and device checks

```
npx expo install expo-notifications expo-device expo-constants
```

Configure Plugin

Enable notifications in `app.json`

```
1 {  
2   "expo": {  
3     "plugins": [  
4       "expo-notifications"  
5     ]  
6   }  
7 }
```

Get Push Token

After requesting permissions (similar to local notifications), retrieve the unique `ExpoPushToken` for your device and project

- This token enables server-side targeting
 - Without it, pushes can't reach the specific device reliably
- Best Practice
 - Store it securely (e.g., send to your backend on signup)
 - Refresh periodically: Tokens can expire

Get Push Token

Workflow

1. Request permissions
(`Notifications.requestPermissionsAsync()`)
2. Fetch project ID from `expo-constants`
(`Constants.expoConfig.extra.eas.projectId`)
3. Call `Notifications.getExpoPushTokenAsync({
 projectId })`
4. Log or send token to server

Example: Fetch Token

Pseudo-code for push token flow

```
1 import * as Notifications from "expo-notifications";
2 import Constants from "expo-constants";
3
4 async function getPushToken() {
5   // Request permissions (same as local)
6   await Notifications.requestPermissionsAsync();
7
8   // Get token using project ID
9   const projectId = Constants.expoConfig.extra.eas.projectId;
10  const token = await Notifications.getExpoPushTokenAsync({ projectId });
11
12  console.log("Token:", token.data); // Send this to your server
13  return token.data;
14 }
15
16 // Call on app load: getPushToken();
```

Send from Server

Your backend uses the push token to send notifications via Expo's cloud service (the "Expo Push Service")

- Expo routes them reliably to device (even if closed)

Simple Flow

1. Server receives/stores user tokens
2. On event (e.g., new message), POST to Expo's cloud endpoint with token + message
3. Expo's service queues & forwards to device (via FCM/APNs)

Send from Server

Your backend uses the push token to send notifications via Expo's cloud service (the "Expo Push Service")

App → Token to Server → POST to Expo Cloud → Device Notification

Example: Server Send

Pseudo-code for server-side send (e.g., Node.js)

```
1  async function sendPush(token) {
2    const message = {
3      to: token, // User's push token (as "address")
4      sound: "default", // Play sound on arrival
5      title: "New Message",
6      body: "Hello from server!",
7      data: { extra: "info" }, // Custom data for app handling
8    };
9
10   await fetch("https://exp.host/--/api/v2/push/send", {
11     // Expo's cloud endpoint
12     method: "POST",
13     headers: { "Content-Type": "application/json" },
14     body: JSON.stringify(message),
15   });
16 }
17
18 // Usage: sendPush('ExponentPushToken[abc123...]');
```

Tip: Test sending push notifications with **Expo's Online Tool**

- Paste your token for quick delivery via Expo's cloud

Handle Events

Catch arrivals/taps (extends local handlers)

```
// Add listeners (in useEffect)
useEffect(() => {
  const onReceive = Notifications.addNotificationReceivedListener(
    (notification) => {
      console.log("Push arrived:", notification);
      // Update UI/state
    }
  );

  const onTap = Notifications.addNotificationResponseReceivedListener(
    (response) => {
      console.log("Push tapped:", response);
      // Navigate to screen
    }
  );

  return () => {
    onReceive.remove();
    onTap.remove();
  };
});
```

FCM

Firebase Cloud Messaging: Google's free, scalable push notification service

- Handles high-volume sends, topic targeting, analytics
- Common choice for production apps
- Used by Expo under the hood for Android pushes

Documentations

- **Expo: Send Notifications with FCM**: How to integrate FCM directly in Expo for finer-grained control
- **FCM Official Docs**: Full setup, APIs, and tutorials

Push Notifications

Ideal for server-driven, dynamic alerts

Common Use Cases

- **Real-Time Updates:** Chat messages, social feeds
- **Marketing/Promotions:** Sales alerts, app invites
- **Live Events:** News pushes, collaborative apps

Require internet & backend

Scale beyond device limits

Lecture Summary

Handling Async Side Effects in Redux

Keep reducers synchronous and pure — no `await`, I/O, API calls, etc.

Offload async logic to

- **Thunks:** Action creators that return functions
- **Middleware:** Automatic side-effect handlers

Lecture Summary

Handling Async Side Effects in Redux

- **Thunks:** Action creators that return functions
 - Receive `dispatch` (and optionally `getState`)
 - Perform async work and dispatch actions before/after
- **Middleware:** Automatic side-effect handlers
 - Observes actions as they are dispatched and triggers side effects automatically

Thunks for explicit, user-triggered async

Middleware for automatic, action-based effects

Lecture Summary

Notifications: Enhance user engagement with timely alerts

- **Local Notifications:** Device-scheduled, offline
 - Install `expo-notifications` to use Expo Notifications
 - Set handler; request permissions; schedule; listen to user taps
- **Push Notifications:** Server-delivered, internet-required
 - Require backend + provider (FCM, APNs, Expo)
 - Get push token for targeting; store securely on backend

Speaker notes